

Федеральное автономное образовательное учреждение высшего образования «Национальный исследовательский университет ИТМО»

Факультет Программной инженерии и компьютерной техники

Дисциплина: «Вычислительная математика»

Лабораторная работа №4

«Методы аппроксимации»

Вариант 9

Выполнила:
Студентка группы Р3213 Пименова Екатерина Андреевна

Преподаватель:
Машина Екатерина Алексеевна

г. Санкт-Петербург
2025

ОГЛАВЛЕНИЕ

ЦЕЛЬ РАБОТЫ	3
ВАРИАНТ 9.....	3
РАСЧЕТЫ ВРУЧНУЮ	4
ОПИСАНИЕ ИСХОДНЫХ ДАННЫХ.....	7
ПРИМЕР 1	7
ПРИМЕР 2	7
ПРИМЕР 3	7
КОД.....	8
MAIN.PY	8
APPROXIMATION_METHODS.PY.....	11
GRAPHICS_DRAWER.PY	14
TABLE_PRINTER.PY.....	18
РЕЗУЛЬТАТЫ РАБОТЫ ПРОГРАММЫ.....	19
ПРИМЕР 1	19
ПРИМЕР 2	21
ПРИМЕР 3	22
ВЫВОД	23

Цель работы

Цель работы – изучение численных методов аппроксимации, ручной расчет и их реализация на ЭВМ.

Вариант 9

Вычислительная реализация задачи

Вычислительная часть лабораторной работы должна быть представлена только в отчете.

Задание:

1. Сформировать таблицу табулирования заданной функции на указанном интервале (см. табл. 1)
2. Построить линейное и квадратичное приближения по 11 точкам заданного интервала;
3. Найти среднеквадратические отклонения для каждой аппроксимирующей функции. Ответы дать с тремя знаками после запятой;
4. Выбрать наилучшее приближение;
5. Построить графики заданной функции, а также полученные линейное и квадратичное приближения;
6. Привести в отчете **подробные вычисления.**

Программная реализация задачи

Для исследования использовать:

- линейную функцию,
- полиномиальную функцию 2-й степени,
- полиномиальную функцию 3-й степени,
- экспоненциальную функцию,
- логарифмическую функцию,
- степенную функцию.

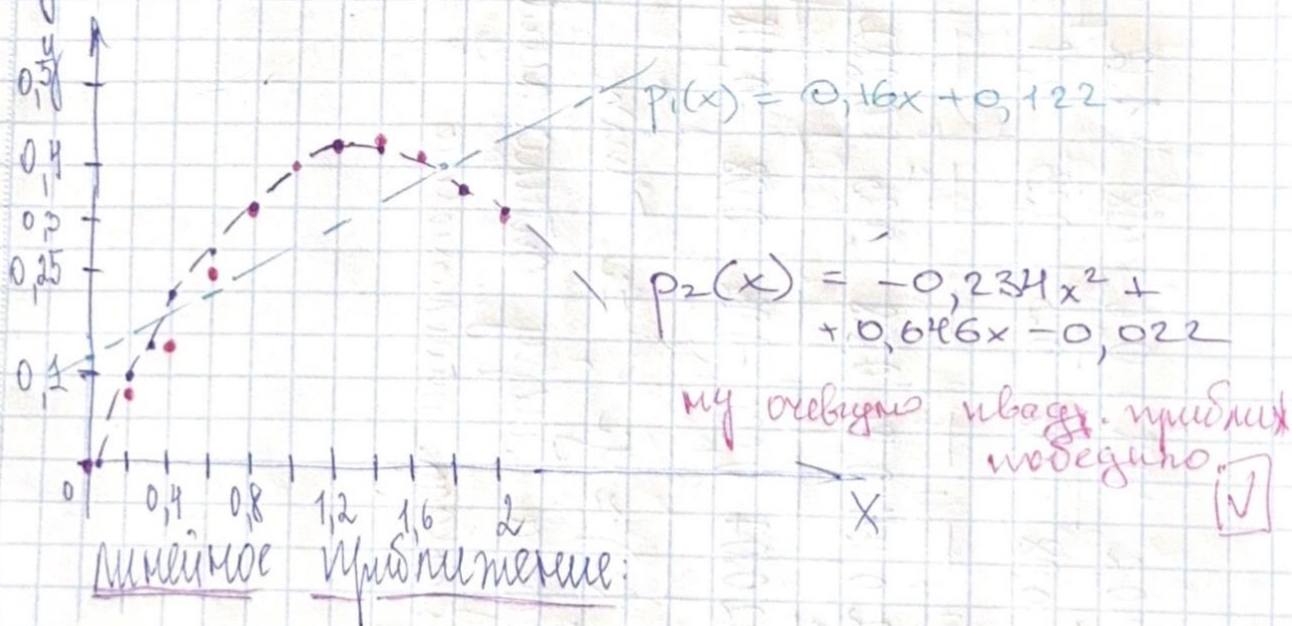
Расчеты вручную

Лаб. работа №4
Вычисл. часть
вариант 9

$$y = \frac{f_x}{x^4 + 9} \quad x \in [0, 2] \quad h = 0,2$$

~~| | | | | | | | | | | | |
|---|---|------|------|------|------|-----|------|------|------|------|------|
| x | 0 | 0,2 | 0,4 | 0,6 | 0,8 | 1 | 1,2 | 1,4 | 1,6 | 1,8 | 2 |
| y | 0 | 0,08 | 0,18 | 0,25 | 0,33 | 0,4 | 0,42 | 0,44 | 0,41 | 0,37 | 0,32 |~~

x	0	0,2	0,4	0,6	0,8	1	1,2	1,4	1,6	1,8	2
y	0	0,08	0,18	0,25	0,33	0,4	0,42	0,44	0,41	0,37	0,32



$$\begin{aligned} \sum x &= \sum_{i=1}^n x_i = \frac{2}{2} \cdot 11 = 11 & \sum y &= \sum_{i=1}^n y_i = 3,24 \\ \sum x^2 &= \sum_{i=1}^n x_i^2 = 15,4 & \sum xy &= \sum_{i=1}^n x_i y_i = 11,0 \end{aligned}$$

получим сист. уравн. отн. a и b

$$\begin{cases} a SXX + b SX = SY \\ a SX + b n = SY \end{cases}$$

из которой по правилу Крамера:

$$\Delta = SXX \cdot n - SX \cdot SX = 15,4 \cdot 11 - 11^2 = 48,4$$

$$\Delta_1 = SY \cdot n - SX \cdot SY = 4 \cdot 11 - 11 \cdot 3,24 = 8,36$$

$$\Delta_2 = SXX \cdot SY - SX \cdot SXY = 15,4 \cdot 3,24 - 11 \cdot 4 = 5,896$$

$$a = \frac{\Delta_1}{\Delta} \quad b = \frac{\Delta_2}{\Delta}$$

$$a = \frac{8,36}{48,4} = 0,16 \quad b = \frac{5,896}{48,4} = 0,122$$

проверим правильность выбора миним. модели:

$$P_1(x) = 0,16x + 0,122$$

x	0	0,2	0,4	0,6	0,8	1	1,2	1,4	1,6	1,8	2
y	0	0,08	0,18	0,25	0,33	0,4	0,42	0,44	0,41	0,37	0,32
$P_1(x)$	0,122	0,154	0,186	0,218	0,25	0,282	0,314	0,346	0,378	0,41	0,442
ε_i	0,122	0,07	0,006	-0,02	-0,08	-0,118	-0,106	-0,094	-0,032	0,04	0,122

$$P_1(x_i) \approx y_i; \quad \varepsilon_i \rightarrow \min$$

определим меру отклонения: $S = \sum_{i=1}^n \varepsilon_i^2 = 0,07$

среднеквадр. отклонение $\delta = \sqrt{\frac{\sum \varepsilon_i^2}{n}} = \sqrt{\frac{0,07}{11}} = 0,08$
милли.

квадратичная аппроксимация
решить систему:

$$\begin{aligned} \sum x_i &= 11 & \sum x_i^2 &= 15,4 & \sum x_i^3 &= 24,2 \\ \sum y_i &= 3,24 & \sum x_i y_i &= 4,01 & \sum x_i^2 y_i &= 5,75296 \\ \sum x_i^4 &= 40,5328 \end{aligned}$$

$$\begin{cases} a_0 \cdot 11 + a_1 \cdot 11 + a_2 \cdot 15,4 = 3,24 \\ a_0 \cdot 11 + a_1 \cdot 15,4 + a_2 \cdot 24,2 = 4,01 \\ a_0 \cdot 15,4 + a_1 \cdot 24,2 + a_2 \cdot 40,5328 = 5,75296 \end{cases}$$

$$a_0 = \frac{-619}{28600} \approx -0,022$$

$$a_1 = \frac{3693}{5720} \approx 0,646$$

$$a_2 = \frac{-673}{2860} \approx -0,234$$

$$P_2(x) = -0,234x^2 + 0,646x - 0,022$$

x	0	0,2	0,4	0,6	0,8	1	1,2	1,4	1,6	1,8	2
y	0	0,08	0,18	0,25	0,33	0,4	0,42	0,44	0,41	0,37	0,32
P ₂ (x)	-0,022	0,098	0,199	0,27	0,34	0,39	0,42	0,42	0,42	0,37	0,33
ε _i	-0,022	0,009	0,02	0,02	0,01	-0,01	0	-0,02	0,01	0	0,01

$$P_2(x_i) \rightarrow y_i; \quad \varepsilon_i \rightarrow \min$$

$$\text{определим меру откл.: } \sum \varepsilon_i^2 = 0,001 \quad \checkmark$$

$$\text{среднеквадратичное отклонение } \sqrt{\frac{\sum \varepsilon_i^2}{n}} = \sqrt{\frac{0,001}{11}} =$$

$$0,008$$

всего миллим

Описание исходных данных

Пользователь вводит с клавиатуры:

- выбор формата ввода данных:

- символ "f", если хочет прочитать таблицу значений из файла. Следующим запросом вводит название файла;
- любой другой символ в случае ввода данных вручную, следующим запросом – кол-во узлов функции (от 8 до 12); следующий запрос – значения x_i через пробел; следующий - y_i через пробел;

- выбор формата вывода результатов:

- символ "f", если хочет записать результаты в файл output.txt;
- любой другой символ, если хочет получить вывод в консоль.

пример 1

```
/Users/sergejpimenov/PycharmProjects/vychmat/venv/bin/python /Users/sergejpimenov/PycharmProjects/vychmat/venv/bin/python
введите f если хотите прочитать из файла и любой другой символ в ином случае:f
введите название файла:input.txt
введите f, если хотите записать результаты в файл и любой другой символ, если в консоль):
```

пример 2

```
/Users/sergejpimenov/PycharmProjects/vychmat/venv/bin/python /Users/sergejpimenov/PycharmProjects/vychmat/venv/bin/python
введите f если хотите прочитать из файла и любой другой символ в ином случае:
введите количество узлов функции:11
введите через пробел значения x_i:-4 -3.6 -3.2 -2.8 -2.4 -2 -1.6 -1.2 -0.8 -0.4 0
введите через пробел значения y_i:-0.25 -0.33 -0.45 -0.61 -0.82 -1.06 -1.2 -1.12 -0.82 -0.42 0
```

пример 3

```
/Users/sergejpimenov/PycharmProjects/vychmat/venv/bin/python /Users/sergejpimenov/PycharmProjects/vychmat/venv/bin/python
введите f если хотите прочитать из файла и любой другой символ в ином случае:
введите количество узлов функции:12
введите через пробел значения x_i:аольр
введите через пробел значения y_i:3333
error: некорректный ввод :( попробуйте еще раз!
введите f если хотите прочитать из файла и любой другой символ в ином случае:|
```

КОД

main.py

```
from approximation_methods import *
from graphics_drawer import *
from table_printer import *

# мапа полученных значений коэфф
results = {"linear": [None, None],
           "quadratic": [None] * 3,
           "cubic": [None] * 4,
           "exp": [None] * 4,
           "log": [None] * 4,
           "pow": [None] * 4
          }

def validate_data(n, x_line, y_line):
    if n < 8 or n > 12:
        raise ValueError
    X = [float(x) for x in x_line.split()]
    if len(X) != n:
        raise ValueError
    Y = [float(y) for y in y_line.split()]
    if len(Y) != n:
        raise ValueError
    return n, X, Y

def read_data():
    while True:
        try:
            key = input("введите f если хотите прочитать из файла и
любой другой символ в ином случае:")
            if key.strip() == "f":
                with open(input("введите название файла:")) as f:
                    n = int(f.readline())
                    x_line = f.readline().replace(",", ".")
                    y_line = f.readline().replace(",", ".")
            else:
                n = int(input("введите количество узлов функции:"))
                x_line = input("введите через пробел значения x_i:")
                y_line = input("введите через пробел значения y_i:")
            n, X, Y = validate_data(n, x_line, y_line)
            return n, X, Y
        except ValueError:
            print("error: некорректный ввод :( попробуйте еще раз!")
            continue

def print_data():
    key = input("введите f, если хотите записать результаты в файл и
```



```

любой другой символ, если в консоль):")
    if key.strip() == "f":
        with open("output.txt", mode="w") as f:
            print("МЕТОД ЛИНЕЙНОЙ АППРОКСИМАЦИИ:".center(90), file=f)
            print_table(n, X, Y, lambda x: results["linear"][0] * x +
results["linear"][1], f)

            print("МЕТОД КВАДРАТИЧНОЙ АППРОКСИМАЦИИ:".center(90),
file=f)
            print_table(n, X, Y,
                        lambda x: results["quadratic"][0] * x ** 2 +
results["quadratic"][1] * x + results["quadratic"][
2], f)

            print("МЕТОД КУБИЧЕСКОЙ АППРОКСИМАЦИИ:".center(90), file=f)
            print_table(n, X, Y, lambda x:
results["cubic"][0] * x ** 3 + results["cubic"][1] * x ** 2
+ results["cubic"][2] * x + results["cubic"][3],
f)

            if all(x is not None for x in results["exp"]):
                print("МЕТОД ЭКСПОНЕНЦИАЛЬНОЙ АППРОКСИМАЦИИ".center(90),
file=f)
                print_table(len(results["exp"][2]), results["exp"][2],
results["exp"][3],
                        lambda x: results["exp"][0] *
math.exp(results["exp"][1] * x), f)

                if all(x is not None for x in results["log"]):
                    print("МЕТОД ЛОГАРИФИЧЕСКОЙ АППРОКСИМАЦИИ".center(90),
file=f)
                    print_table(len(results["log"][2]), results["log"][2],
results["log"][3],
                                lambda x: results["log"][0] * math.log(x) +
results["log"][1], f)

                    if all(x is not None for x in results["pow"]):
                        print("МЕТОД СТЕПЕННОЙ АППРОКСИМАЦИИ".center(90),
file=f)
                        print_table(len(results["pow"][2]), results["pow"][2],
results["pow"][3],
                                lambda x: results["log"][0] * x **
results["log"][1], f)

                        return

            print("МЕТОД ЛИНЕЙНОЙ АППРОКСИМАЦИИ:".center(90))
            print_table(n, X, Y, lambda x: results["linear"][0] * x +
results["linear"][1], None)

            print("МЕТОД КВАДРАТИЧНОЙ АППРОКСИМАЦИИ:".center(90))
            print_table(n, X, Y,
                        lambda x: results["quadratic"][0] * x ** 2 +
results["quadratic"][1] * x + results["quadratic"][2],
                        None)

```

```

print("МЕТОД КУБИЧЕСКОЙ АППРОКСИМАЦИИ:".center(90))
print_table(n, X, Y, lambda x:
    results["cubic"][0] * x ** 3 + results["cubic"][1] * x ** 2 +
    results["cubic"][2] * x + results["cubic"][3], None)

    if all(x is not None for x in results["exp"]):
        print("МЕТОД ЭКСПОНЕНЦИАЛЬНОЙ АППРОКСИМАЦИИ".center(90))
        print_table(len(results["exp"][2]), results["exp"][2],
results["exp"][3],
                    lambda x: results["exp"][0] *
math.exp(results["exp"][1] * x),
                    None)

    if all(x is not None for x in results["log"]):
        print("МЕТОД ЛОГАРИФМИЧЕСКОЙ АППРОКСИМАЦИИ".center(90))
        print_table(len(results["log"][2]), results["log"][2],
results["log"][3],
                    lambda x: results["log"][0] * math.log(x) +
results["log"][1], None)

    if all(x is not None for x in results["pow"]):
        print("МЕТОД СТЕПЕННОЙ АППРОКСИМАЦИИ".center(90))
        print_table(len(results["pow"][2]), results["pow"][2],
results["pow"][3],
                    lambda x: results["log"][0] * x **
results["log"][1], None)

if __name__ == "__main__":
    n, X, Y = read_data()

    a, b = linear_approximation(n, X, Y)
    results["linear"] = a, b
    draw_linear_graph(X, Y, a, b)

    a, b, c = quadratic_approximation(n, X, Y)
    results["quadratic"] = float(a), float(b), float(c)
    draw_quadratic_graph(X, Y, a, b, c)

    a, b, c, d = cubic_approximation(n, X, Y)
    results["cubic"] = float(a), float(b), float(c), float(d)
    draw_cubic_approximation(X, Y, a, b, c, d)

    try:
        a, b, accepted_X, accepted_Y = exponential_approximation(n, X,
Y)
        results["exp"] = float(a), float(b), accepted_X, accepted_Y
        draw_exp_approximation(X, Y, accepted_X, a, b)
    except TypeError:
        print(
            "error: ОДЗ логарифма x>0 -> недостаточно точек с
положительными значениями функции для аппроксимации данным методом")

    try:
        a, b, accepted_X, accepted_Y = logarithm_approximation(n, X, Y)

```

```

        results["log"] = float(a), float(b), accepted_X, accepted_Y
        draw_log_approximation(X, Y, accepted_X, a, b)

    except TypeError:
        print(
            "error: ОДЗ логарифма  $x > 0$  -> недостаточно точек с
положительными значениями функции для аппроксимации данным методом")

    try:
        a, b, accepted_X, accepted_Y = pow_approximation(n, X, Y)
        results["pow"] = float(a), float(b), accepted_X, accepted_Y
        draw_pow_approximation(X, Y, accepted_X, a, b)

    except TypeError:
        print(
            "error: ОДЗ логарифма  $x > 0$  -> недостаточно точек с
положительными значениями функции для аппроксимации данным методом")

    print_data()
    draw_all_approximations(X, Y, results)

```

approximation_methods.py

```

import math

import numpy as np
from math import exp, log

def count_sums_1(n, X, Y):
    S_X = sum(X)
    S_Y = sum(Y)
    S_XX = sum(map(lambda x: x ** 2, X))
    S_XY = sum([X[i] * Y[i] for i in range(n)])
    return S_X, S_Y, S_XX, S_XY

def count_sums_2(n, X, Y):
    S_X3 = sum(map(lambda x: x ** 3, X))
    S_X2Y = sum([X[i] ** 2 * Y[i] for i in range(n)])
    S_X4 = sum(map(lambda x: x ** 4, X))
    return S_X3, S_X2Y, S_X4

def count_sums_3(n, X, Y):
    S_X5 = sum(map(lambda x: x ** 5, X))
    S_X3Y = sum([X[i] ** 3 * Y[i] for i in range(n)])
    S_X6 = sum(map(lambda x: x ** 6, X))
    return S_X5, S_X6, S_X3Y

def linear_approximation(n, X, Y) -> (float, float):
    """
    :return: коэффициенты  $a$  и  $b$  аппроксимирующей функции  $ax+b$ 
    """

```

```

S_X, S_Y, S_XX, S_XY = count_sums_1(n, X, Y)

# получим сист лин уравн и по правилу Крамера посчитаем
delta = S_XX * n - S_X ** 2
delta_1 = S_XY * n - S_X * S_Y
delta_2 = S_XX * S_Y - S_X * S_XY

return delta_1 / delta, delta_2 / delta

def solve_linear_system(A, B):
    A = np.array(A)
    B = np.array(B)
    try:
        solution = np.linalg.solve(A, B)
        return solution
    except np.linalg.LinAlgError:
        return None, None

def quadratic_approximation(n, X, Y) -> (float, float, float):
    """
    :return: коэффициенты a b c аппроксимирующей функции  $ax^2 + bx + c$ 
    """
    S_X, S_Y, S_X2, S_XY = count_sums_1(n, X, Y)
    S_X3, S_X2Y, S_X4 = count_sums_2(n, X, Y)
    A = [[n, S_X, S_X2],
          [S_X, S_X2, S_X3],
          [S_X2, S_X3, S_X4]]
    B = [S_Y, S_XY, S_X2Y]

    a, b, c = solve_linear_system(A, B)[::-1]

    return a, b, c

def cubic_approximation(n, X, Y) -> (float, float, float, float):
    """
    :return: коэффициенты a b c d аппроксимирующей функции  $ax^3 + bx^2 + cx + d$ 
    """
    S_X, S_Y, S_X2, S_XY = count_sums_1(n, X, Y)
    S_X3, S_X2Y, S_X4 = count_sums_2(n, X, Y)
    S_X5, S_X6, S_X3Y = count_sums_3(n, X, Y)
    A = [
        [S_X6, S_X5, S_X4, S_X3],
        [S_X5, S_X4, S_X3, S_X2],
        [S_X4, S_X3, S_X2, S_X],
        [S_X3, S_X2, S_X, n]
    ]

    B = [S_X3Y, S_X2Y, S_XY, S_Y]
    return solve_linear_system(A, B)

```



```

def exponential_approximation(n, X, Y):
    """
    :return: коэфф a, b функции  $ae^{bx}$ 
    """
    clean_X = []
    clean_Y = []

    for x, y in zip(X, Y):
        if y is not None and y > 0: # Явная проверка
            clean_X.append(x)
            clean_Y.append(y)

    if len(clean_Y) < 2: # Для линейной регрессии нужно минимум 2 точки
        return None, None, None, None

    try:
        lnY = [math.log(y) for y in clean_Y] # Более питонический
способ
        B, A = linear_approximation(len(lnY), clean_X, lnY)
        return math.exp(A), B, clean_X, clean_Y
    except (ValueError, TypeError):
        return None, None, None, None

def logarithm_approximation(n, X, Y):
    clean_X = []
    clean_Y = []

    for x, y in zip(X, Y):
        if x is not None and y is not None and x > 0: # ln(x) определен
только для x > 0
            clean_X.append(x)
            clean_Y.append(y)

    if len(clean_X) < 2:
        return None, None, None, None

    try:
        # линеаризация - заменяем x на ln(x)
        lnX = [math.log(x) for x in clean_X]

        # y = a*lnX + b
        a, b = linear_approximation(len(lnX), lnX, clean_Y)

        return a, b, clean_X, clean_Y

    except (ValueError, TypeError, ZeroDivisionError):
        return None, None, None, None

def pow_approximation(n, X, Y):
    clean_X = []
    clean_Y = []

    for x, y in zip(X, Y):
        if x is not None and y is not None and x > 0 and y > 0:

```

```

        clean_X.append(x)
        clean_Y.append(y)

if len(clean_X) < 2:
    return None, None, None, None

try:
    # Линеаризация
    lnX = [math.log(x) for x in clean_X]
    lnY = [math.log(y) for y in clean_Y]

    # Линейная регрессия:  $\ln Y = \ln(a) + b \cdot \ln X$ 
    b, ln_a = linear_approximation(len(lnX), lnX, lnY)
    a = math.exp(ln_a)

    return a, b, clean_X, clean_Y

except (ValueError, TypeError, ZeroDivisionError):
    return None, None, None, None

```

graphics_drawer.py

```

import numpy as np
import matplotlib.pyplot as plt
from math import exp

def draw_linear_graph(X, Y, a, b):
    plt.figure(figsize=(10, 6))

    # Рисуем точки
    plt.scatter(X, Y, color='pink', label='исходные точки (X, Y)',
                zorder=5)

    # Рисуем линейную функцию
    x_line = np.linspace(min(X) - 0.1, max(X) + 0.1, 100)
    y_line = a * x_line + b
    plt.plot(x_line, y_line, label=f'аппроксимирующая функция:  $y = \{round(a, 3)\}x + \{round(b, 3)\}$ ', color='darkcyan')

    plt.title('')
    plt.xlabel('Ox')
    plt.ylabel('Oy')
    plt.grid(True, linestyle='--', alpha=0.7)
    plt.legend()
    plt.tight_layout()

    plt.savefig('graphics/linear_approximation.png')
    plt.close()

def draw_quadratic_graph(X, Y, a, b, c):
    plt.figure(figsize=(10, 6))

```

```

# Рисуем точки
plt.scatter(X, Y, color='pink', label='исходные точки (X, Y)',
zorder=5)
x_line = np.linspace(min(X) - 0.1, max(X) + 0.1, 100)
y_line = a * x_line**2 + b*x_line + c
plt.plot(x_line, y_line, label=f'аппроксимирующая функция:  $y =$ 
{round(a, 3)}x^2 + {round(b, 3)}x + {round(c, 3)}', color='lightseagreen')

plt.title('')
plt.xlabel('Ox')
plt.ylabel('Oy')
plt.grid(True, linestyle='--', alpha=0.7)
plt.legend()
plt.tight_layout()
plt.savefig('graphics/quadratic_approximation.png')
plt.close()

def draw_cubic_approximation(X, Y, a, b, c, d):
    plt.figure(figsize=(10, 6))

    # Рисуем точки
    plt.scatter(X, Y, color='pink', label='исходные точки (X, Y)',
zorder=5)
    x_line = np.linspace(min(X) - 0.1, max(X) + 0.1, 100)
    y_line = a * x_line ** 3 + + b * x_line**2 + c*x_line + d
    plt.plot(x_line, y_line, label=f'аппроксимирующая функция:  $y =$ 
{round(a, 3)}x^3 + {round(b, 3)}x^2 + {round(c, 3)}x + {round(d, 3)}',
color='lightseagreen')

    plt.title('')
    plt.xlabel('Ox')
    plt.ylabel('Oy')
    plt.grid(True, linestyle='--', alpha=0.7)
    plt.legend()
    plt.tight_layout()
    plt.savefig('graphics/cubic_approximation.png')
    plt.close()

def draw_exp_approximation(X, Y, acc_X, a, b):
    plt.figure(figsize=(10, 6))

    plt.scatter(X, Y, color='pink', label='исходные точки (X, Y)',
zorder=5)
    x_line = np.linspace(min(acc_X) - 0.1, max(acc_X) + 0.1, 100)
    y_line = a * np.exp(b*x_line)
    plt.plot(x_line, y_line,
label=f'аппроксимирующая функция:  $y =$  {round(a, 3)}*
e^{round(b, 3)}x',
color='lightseagreen')

    plt.title('')
    plt.xlabel('Ox')
    plt.ylabel('Oy')
    plt.grid(True, linestyle='--', alpha=0.7)

```

```

plt.legend()
plt.tight_layout()
plt.savefig('graphics/exp_approximation.png')
plt.close()

def draw_log_approximation(X, Y, acc_x, a, b):
    plt.figure(figsize=(10, 6))

    plt.scatter(X, Y, color='pink', label='исходные точки (X, Y)',
zorder=5)
    x_line = np.linspace(min(acc_x) - 0.1, max(acc_x) + 0.1, 100)
    y_line = a * np.log(x_line) + b
    plt.plot(x_line, y_line,
            label=f'аппроксимирующая функция:  $y = \{round(a, 3)\} \cdot \ln(x) + \{round(b, 3)\}$ ',
            color='lightseagreen')

    plt.title('')
    plt.xlabel('Ox')
    plt.ylabel('Oy')
    plt.grid(True, linestyle='--', alpha=0.7)
    plt.legend()
    plt.tight_layout()
    plt.savefig('graphics/log_approximation.png')
    plt.close()

def draw_pow_approximation(X, Y, acc_x, a, b):
    plt.figure(figsize=(10, 6))

    plt.scatter(X, Y, color='pink', label='исходные точки (X, Y)',
zorder=5)
    x_line = np.linspace(min(acc_x) - 0.1, max(acc_x) + 0.1, 100)
    y_line = a * x_line**b
    plt.plot(x_line, y_line,
            label=f'аппроксимирующая функция:  $y = \{round(a, 3)\} x^{\{round(b, 3)\}}$ ',
            color='lightseagreen')

    plt.title('')
    plt.xlabel('Ox')
    plt.ylabel('Oy')
    plt.grid(True, linestyle='--', alpha=0.7)
    plt.legend()
    plt.tight_layout()
    plt.savefig('graphics/pow_approximation.png')
    plt.close()

def draw_all_approximations(X, Y, results):
    linear_coeffs = results["linear"]
    quadratic_coeffs = results["quadratic"]
    cubic_coeffs = results["cubic"]

```

```

exp_coeffs = results["exp"][0:2]
log_coeffs = results["log"][0:2]
pow_coeffs = results["pow"][0:2]
acc_X = results["exp"][2]
plt.figure(figsize=(12, 8))

plt.scatter(X, Y, color='cyan', label='исходные точки (X, Y)',
zorder=10)

x_min = min(X) - 0.5
x_max = max(X) + 0.5
x_line = np.linspace(x_min, x_max, 500)

if linear_coeffs and linear_coeffs[0] is not None:
    a, b = linear_coeffs
    y_linear = a * x_line + b
    plt.plot(x_line, y_linear, label=f'линейн:  $y = \{round(a, 3)\}x + \{round(b, 3)\}$ ',
            color='pink', linestyle='--')

if quadratic_coeffs and all(c is not None for c in
quadratic_coeffs):
    a, b, c = quadratic_coeffs
    y_quad = a * x_line ** 2 + b * x_line + c
    plt.plot(x_line, y_quad, label=f'квадр:  $y = \{round(a, 3)\}x^2 + \{round(b, 3)\}x + \{round(c, 3)\}$ ',
            color='hotpink', linestyle='--')

if cubic_coeffs and all(c is not None for c in cubic_coeffs):
    a, b, c, d = cubic_coeffs
    y_cubic = a * x_line ** 3 + b * x_line ** 2 + c * x_line + d
    plt.plot(x_line, y_cubic,
            label=f'кубич:  $y = \{round(a, 3)\}x^3 + \{round(b, 3)\}x^2 + \{round(c, 3)\}x + \{round(d, 3)\}$ ',
            color='Violet', linestyle='--')

if exp_coeffs and all(c is not None for c in exp_coeffs[:2]):
    a, b = exp_coeffs[:2]
    y_exp = a * np.exp(b * x_line)
    plt.plot(x_line, y_exp, label=f'экспоненц:  $y = \{round(a, 3)\}e^{\{round(b, 3)\}x}$ ',
            color='grey', linestyle='--')

if log_coeffs and all(c is not None for c in log_coeffs[:2]):
    a, b = log_coeffs[:2]
    x_log = x_line[x_line > 0.01] # Избегаем log(0)
    y_log = a * np.log(x_log) + b
    plt.plot(x_log, y_log, label=f'лог:  $y = \{round(a, 3)\}\ln(x) + \{round(b, 3)\}$ ',
            color='lightgrey', linestyle='--')

if pow_coeffs and all(c is not None for c in pow_coeffs[:2]):
    a, b = pow_coeffs[:2]
    x_pow = x_line[x_line > 0.01] # Избегаем 0^b
    y_pow = a * x_pow ** b

```



```

plt.plot(x_pow, y_pow, label=f'степен: y = {round(a,
3)}x^{round(b, 3)}',
         color='DarkSlateGrey', linestyle='--')

plt.title('сравнение методов аппроксимации')
plt.xlabel('Ox')
plt.ylabel('Oy')
plt.grid(True, linestyle='--', alpha=0.5)
plt.legend(bbox_to_anchor=(1.05, 1), loc='upper right')
plt.tight_layout()

plt.savefig('graphics/all_approximations.png', bbox_inches='tight',
dpi=300)
plt.show()

```

table_printer.py

```

import math

def print_table(n, X, Y, phi, output_file):
    print("X".center(10) + "|" + "|".join([str(round(x, 4)).center(12)
for x in X]), file=output_file)
    print("-" * (n + 1) * 13, file=output_file)
    print("Y".center(10) + "|" + "|".join([str(round(y, 4)).center(12)
for y in Y]), file=output_file)
    print("-" * (n + 1) * 13, file=output_file)

    print("phi(X)".center(10) + "|" + "|".join([str(round(phi(x),
4)).center(12) for x in X]), file=output_file)
    print("-" * (n + 1) * 13, file=output_file)

    e_i = [phi(X[i]) - Y[i] for i in range(n)]
    print("e_i".center(10) + "|" + "|".join([str(round(e, 4)).center(12)
for e in e_i]), file=output_file)

    e_i2 = list(map(lambda x: x ** 2, e_i))
    print(f"СРЕДНЕКВАДРАТИЧНОЕ ОТКЛОНЕНИЕ: {round(math.sqrt(sum(e_i2) /
n), 4)}", file=output_file)
    print("\n", file=output_file)

```

РЕЗУЛЬТАТЫ РАБОТЫ ПРОГРАММЫ

ПРИМЕР 1

```
/Users/sergejpimenov/PycharmProjects/vychmat/venv/bin/python /Users/sergejpimenov/PycharmProjects/vychmat/lab4/main.py
введите f если хотите прочитать из файла и любой другой символ в ином случае:f
введите название файла:input.txt
введите f, если хотите записать результаты в файл и любой другой символ, если в консоль):

МЕТОД ЛИНЕЙНОЙ АППРОКСИМАЦИИ:
X | 0.0 | 0.2 | 0.4 | 0.6 | 0.8 | 1.0 | 1.2 | 1.4 | 1.6 | 1.8 | 2.0
Y | 0.0 | 0.0889 | 0.1773 | 0.2629 | 0.3401 | 0.4 | 0.4335 | 0.4361 | 0.4115 | 0.3693 | 0.32
phi(X) | 0.1189 | 0.154 | 0.1891 | 0.2242 | 0.2594 | 0.2945 | 0.3296 | 0.3647 | 0.3999 | 0.435 | 0.4701
e_i | 0.1189 | 0.0651 | 0.0118 | -0.0386 | -0.0807 | -0.1055 | -0.1038 | -0.0713 | -0.0116 | 0.0657 | 0.1501
СРЕДНЕКВАДРАТИЧНОЕ ОТКЛОНЕНИЕ: 0.0856

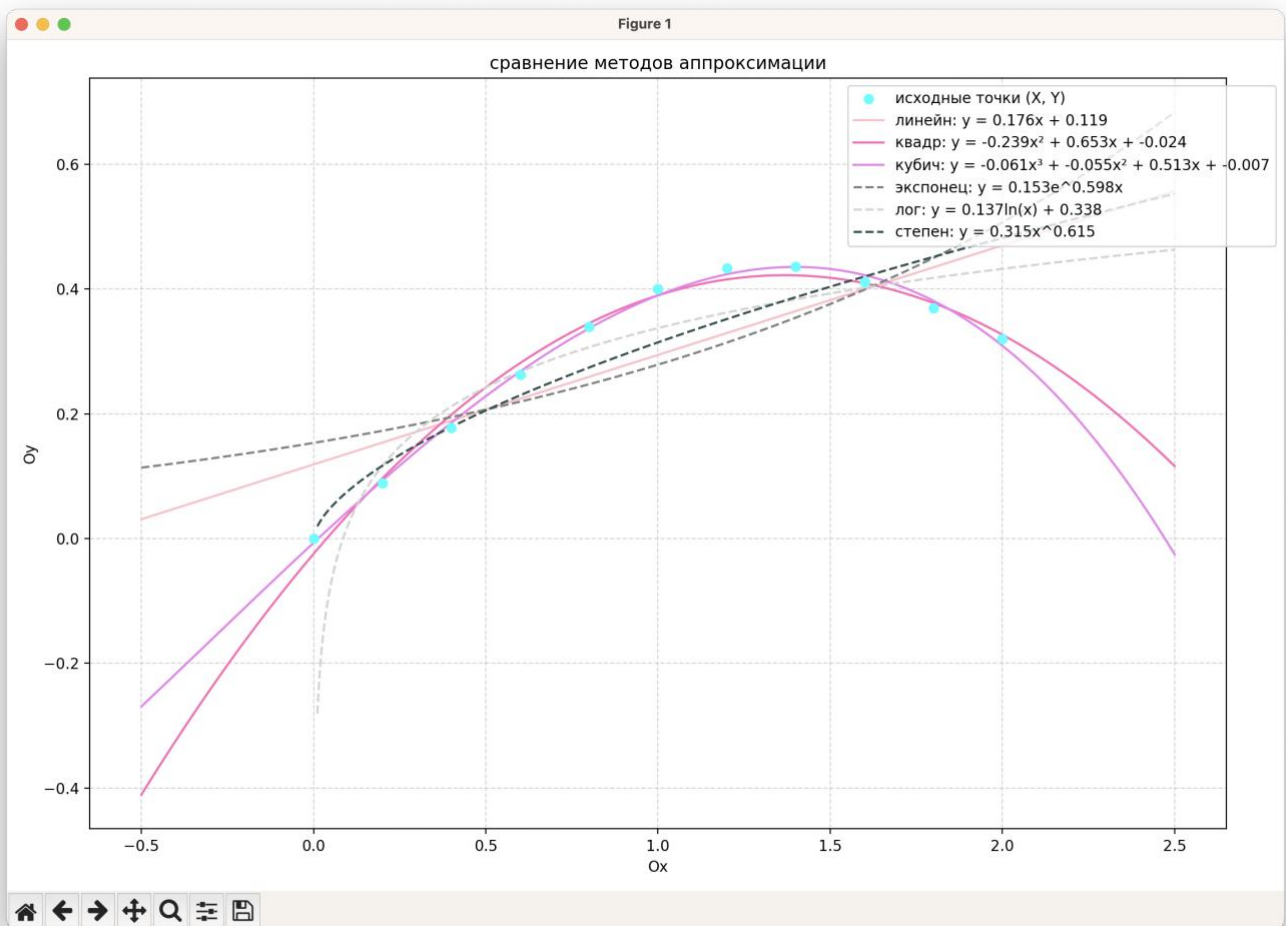
МЕТОД КВАДРАТИЧНОЙ АППРОКСИМАЦИИ:
X | 0.0 | 0.2 | 0.4 | 0.6 | 0.8 | 1.0 | 1.2 | 1.4 | 1.6 | 1.8 | 2.0
Y | 0.0 | 0.0889 | 0.1773 | 0.2629 | 0.3401 | 0.4 | 0.4335 | 0.4361 | 0.4115 | 0.3693 | 0.32
phi(X) | -0.0244 | 0.0967 | 0.1987 | 0.2815 | 0.3453 | 0.39 | 0.4156 | 0.4221 | 0.4094 | 0.3777 | 0.3269
e_i | -0.0244 | 0.0078 | 0.0214 | 0.0187 | 0.0053 | -0.01 | -0.0179 | -0.014 | -0.0021 | 0.0084 | 0.0069
СРЕДНЕКВАДРАТИЧНОЕ ОТКЛОНЕНИЕ: 0.0142

МЕТОД КУБИЧЕСКОЙ АППРОКСИМАЦИИ:
X | 0.0 | 0.2 | 0.4 | 0.6 | 0.8 | 1.0 | 1.2 | 1.4 | 1.6 | 1.8 | 2.0
Y | 0.0 | 0.0889 | 0.1773 | 0.2629 | 0.3401 | 0.4 | 0.4335 | 0.4361 | 0.4115 | 0.3693 | 0.32
phi(X) | -0.0068 | 0.0931 | 0.1857 | 0.268 | 0.3371 | 0.39 | 0.4238 | 0.4356 | 0.4224 | 0.3812 | 0.3092
e_i | -0.0068 | 0.0043 | 0.0084 | 0.0051 | -0.003 | -0.01 | -0.0096 | -0.0005 | 0.0109 | 0.0119 | -0.0108
СРЕДНЕКВАДРАТИЧНОЕ ОТКЛОНЕНИЕ: 0.0082

МЕТОД ЭКСПОНЕНЦИАЛЬНОЙ АППРОКСИМАЦИИ
X | 0.2 | 0.4 | 0.6 | 0.8 | 1.0 | 1.2 | 1.4 | 1.6 | 1.8 | 2.0
Y | 0.0889 | 0.1773 | 0.2629 | 0.3401 | 0.4 | 0.4335 | 0.4361 | 0.4115 | 0.3693 | 0.32
phi(X) | 0.1729 | 0.1948 | 0.2196 | 0.2475 | 0.2789 | 0.3143 | 0.3543 | 0.3993 | 0.45 | 0.5072
e_i | 0.084 | 0.0175 | -0.0433 | -0.0926 | -0.1211 | -0.1191 | -0.0818 | -0.0122 | 0.0807 | 0.1872
СРЕДНЕКВАДРАТИЧНОЕ ОТКЛОНЕНИЕ: 0.0975

МЕТОД ЛОГАРИФИЧЕСКОЙ АППРОКСИМАЦИИ
X | 0.2 | 0.4 | 0.6 | 0.8 | 1.0 | 1.2 | 1.4 | 1.6 | 1.8 | 2.0
Y | 0.0889 | 0.1773 | 0.2629 | 0.3401 | 0.4 | 0.4335 | 0.4361 | 0.4115 | 0.3693 | 0.32
phi(X) | 0.1168 | 0.2119 | 0.2675 | 0.3069 | 0.3375 | 0.3625 | 0.3836 | 0.402 | 0.4181 | 0.4326
e_i | 0.028 | 0.0346 | 0.0046 | -0.0332 | -0.0625 | -0.071 | -0.0524 | -0.0095 | 0.0488 | 0.1126
СРЕДНЕКВАДРАТИЧНОЕ ОТКЛОНЕНИЕ: 0.0547

МЕТОД СТЕПЕННОЙ АППРОКСИМАЦИИ
X | 0.2 | 0.4 | 0.6 | 0.8 | 1.0 | 1.2 | 1.4 | 1.6 | 1.8 | 2.0
Y | 0.0889 | 0.1773 | 0.2629 | 0.3401 | 0.4 | 0.4335 | 0.4361 | 0.4115 | 0.3693 | 0.32
phi(X) | 0.0796 | 0.1006 | 0.1154 | 0.1272 | 0.1371 | 0.1458 | 0.1536 | 0.1607 | 0.1672 | 0.1733
e_i | -0.0092 | -0.0766 | -0.1475 | -0.2129 | -0.2629 | -0.2876 | -0.2825 | -0.2508 | -0.2021 | -0.1467
СРЕДНЕКВАДРАТИЧНОЕ ОТКЛОНЕНИЕ: 0.2074
```



ПРИМЕР 2

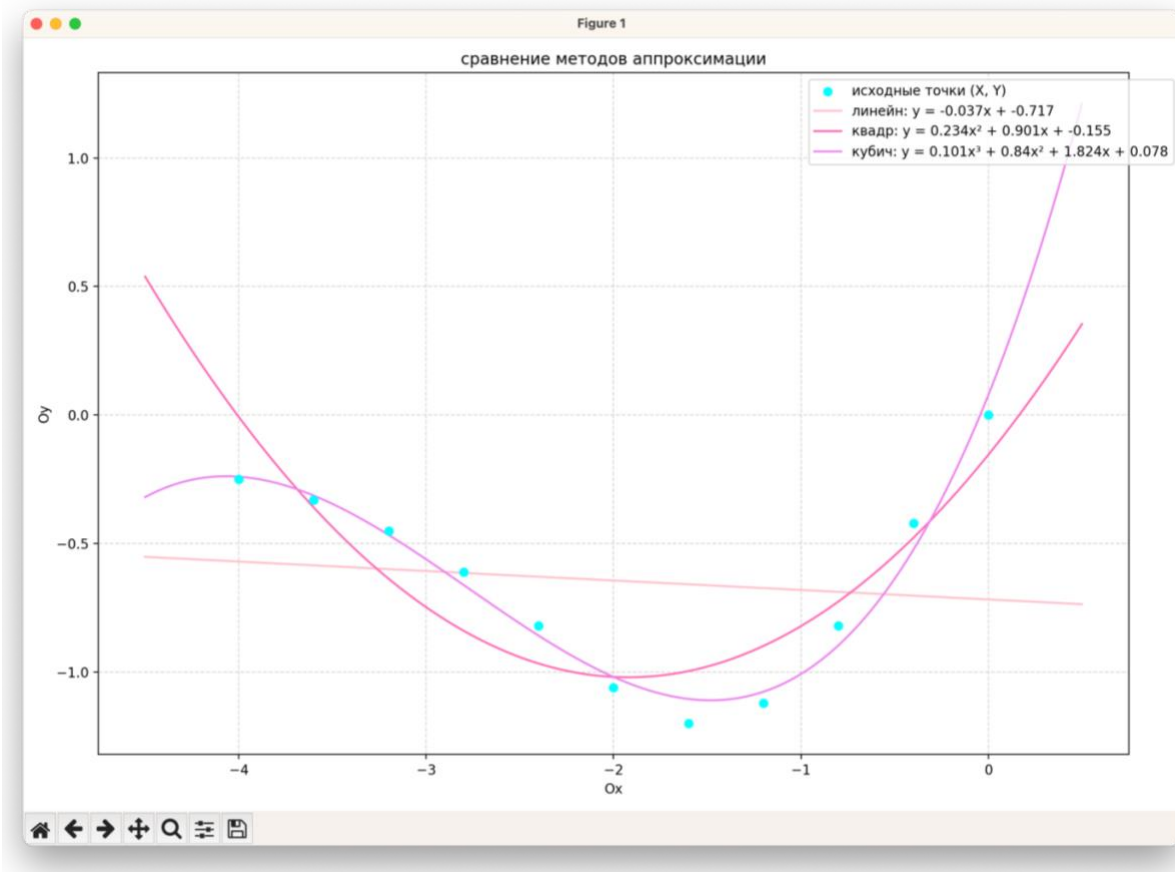
```

/Users/sergejpimenov/PycharmProjects/vychmat/venv/bin/python /Users/sergejpimenov/PycharmProjects/vychmat/lab4/main.py
введите f если хотите прочитать из файла и любой другой символ в ином случае:
введите количество узлов функции:11
введите через пробел значения x_i:-4 -3.6 -3.2 -2.8 -2.4 -2 -1.6 -1.2 -0.8 -0.4 0
введите через пробел значения y_i:-0.25 -0.33 -0.45 -0.61 -0.82 -1.06 -1.2 -1.12 -0.82 -0.42 0
errgr: ОДЗ логарифма x>0 -> недостаточно точек с положительными значениями функции для аппроксимации экспоненциальным методом
errgr: ОДЗ логарифма x>0 -> недостаточно точек с положительными значениями функции для аппроксимации логарифмическим методом
errgr: ОДЗ логарифма x>0 -> недостаточно точек с положительными значениями функции для аппроксимации степенным методом
введите f, если хотите записать результаты в файл и любой другой символ, если в консоль):f

```

содержимое файла output.txt:

1													
2	X	-4.0	-3.6	-3.2	-2.8	-2.4	-2.0	-1.6	-1.2	-0.8	-0.4	0.0	
3													
4	Y	-0.25	-0.33	-0.45	-0.61	-0.82	-1.06	-1.2	-1.12	-0.82	-0.42	0.0	
5													
6	phi(X)	-0.57	-0.5847	-0.5995	-0.6142	-0.6289	-0.6436	-0.6584	-0.6731	-0.6878	-0.7025	-0.7173	
7													
8	e_i	-0.32	-0.2547	-0.1495	-0.0042	0.1911	0.4164	0.5416	0.4469	0.1322	-0.2825	-0.7173	
9	СРЕДНЕКВАДРАТИЧНОЕ ОТКЛОНЕНИЕ: 0.3698												
10													
11													
12													
13	X	-4.0	-3.6	-3.2	-2.8	-2.4	-2.0	-1.6	-1.2	-0.8	-0.4	0.0	
14													
15	Y	-0.25	-0.33	-0.45	-0.61	-0.82	-1.06	-1.2	-1.12	-0.82	-0.42	0.0	
16													
17	phi(X)	-0.0074	-0.3597	-0.637	-0.8392	-0.9665	-1.0187	-0.9959	-0.8981	-0.7253	-0.4775	-0.1547	
18													
19	e_i	0.2426	-0.0297	-0.187	-0.2292	-0.1465	0.0413	0.2041	0.2219	0.0947	-0.0575	-0.1547	
20	СРЕДНЕКВАДРАТИЧНОЕ ОТКЛОНЕНИЕ: 0.1644												
21													
22													
23													
24	X	-4.0	-3.6	-3.2	-2.8	-2.4	-2.0	-1.6	-1.2	-0.8	-0.4	0.0	
25													
26	Y	-0.25	-0.33	-0.45	-0.61	-0.82	-1.06	-1.2	-1.12	-0.82	-0.42	0.0	
27													
28	phi(X)	-0.2398	-0.3132	-0.4666	-0.6611	-0.858	-1.0187	-1.1044	-1.0763	-0.8957	-0.524	0.0777	
29													
30	e_i	0.0102	0.0168	-0.0166	-0.0511	-0.038	0.0413	0.0956	0.0437	-0.0757	-0.104	0.0777	
31	СРЕДНЕКВАДРАТИЧНОЕ ОТКЛОНЕНИЕ: 0.0603												
32													



ПРИМЕР 3

```

/Users/sergejpimenov/PycharmProjects/vychmat/venv/bin/python /Users/sergejpimenov/PycharmProjects/vy
введите f если хотите прочитать из файла и любой другой символ в ином случае:
введите количество узлов функции:kde1
error: некорректный ввод :( попробуйте еще раз!
введите f если хотите прочитать из файла и любой другой символ в ином случае:dfhj
введите количество узлов функции:tyvio
error: некорректный ввод :( попробуйте еще раз!
введите f если хотите прочитать из файла и любой другой символ в ином случае:

```

Вывод

В результате выполнения лабораторной работы номер 4 я изучила численные методы аппроксимации, а также реализовала их на языке программирования python. Мне супер понравилось, реально интересная темка!